

Indoore Backend API Automation Framework

A Layered, Standards-Based Approach to Enterprise Backend Quality Assurance

Volumes 1-5 · Complete Compendium

Document Version 1.0 · July 2026

CORE TECHNOLOGY STACK

Playwright

TypeScript

Node.js

PostgreSQL

Zod

GitHub Actions

Allure Reports

Document Control

Version History

Version	Date	Author	Description
1.0	July 2026	Vamsi Krishna	Initial Enterprise Architecture Handbook

Revision Roadmap

Version	Description
1.0	Framework Foundation
1.1	Core Engine Documentation
1.2	Module Architecture
2.0	Enterprise Release

Document Scope

This handbook consolidates five volumes covering the complete architecture, engineering standards, and operational practices of the Indoore Backend API Automation Framework: framework foundation, core engineering components, the Four Quality Pillars validation strategy, module architecture and catalogue, and CI/CD & engineering operations.

Table of Contents

Volume 1 — Framework Foundation

1	Executive Summary
2	Business Problem
3	Framework Vision
4	Objectives
5	Technology Stack
6	Framework Overview
7	Architecture Principles
8	High-Level Architecture
9	Layered Framework Design
10	Folder Structure
11	Request Lifecycle
12	Framework Features
13	Engineering Achievements
14	Benefits
15	Conclusion

Table of Contents (continued)

Volume 2 — Core Framework Components

1	Core Framework Overview
2	Framework Component Architecture
3	Authentication Engine
4	API Layer
5	Data Layer
6	Mapper Layer
7	Validation Engine
8	Assertion Engine
9	Database Validation Layer
10	Schema Layer (Zod)
11	Shared Utilities
12	Retry & Resilience Engine
13	Error Handling Strategy
14	Logging & Reporting
15	Design Patterns
16	Engineering Benefits
17	Conclusion

Volume 3 — Four Quality Pillars

1	Introduction
2	Why Four Quality Pillars?
3	Quality Architecture
4	Pillar 1 – Zod Schema Validation
5	Pillar 2 – Contract Snapshot Testing
6	Pillar 3 – Mutation-Proof Testing
7	Pillar 4 – Database Cross-Validation
8	Combined Validation Workflow
9	Framework Integration
10	Benefits & Outcomes
11	Best Practices
12	Conclusion

Table of Contents (continued)

Volume 4 — Module Architecture & Module Catalogue

1	Module Architecture Overview
2	Standard Module Design
3	Module Development Lifecycle
4	Common Folder Structure
5	Module Implementation Standards
6	Module Catalogue
7	Module Dependency Matrix
8	Module Coverage Summary
9	Engineering Metrics
10	Best Practices
11	Future Expansion Strategy
12	Conclusion

Volume 5 — CI/CD, Test Execution Pipeline & Engineering Operations

1	Engineering Operations Overview
2	Test Execution Lifecycle
3	Environment Management
4	Git Workflow & Branching Strategy
5	CI/CD Architecture
6	GitHub Actions Pipeline
7	Test Execution Strategy
8	Reporting & Observability
9	Logging & Diagnostics
10	Performance Monitoring
11	Failure Classification & Recovery
12	Release Readiness
13	Operational Best Practices
14	Future Enhancements
15	Conclusion

Framework Foundation

Business context, vision, objectives, technology stack, and the high-level architecture that underpins the Indoore Backend API Automation Framework.

1. Executive Summary
2. Business Problem
3. Framework Vision
4. Objectives
5. Technology Stack
6. Framework Overview
7. Architecture Principles
8. High-Level Architecture
9. Layered Framework Design
10. Folder Structure
11. Request Lifecycle
12. Framework Features
13. Engineering Achievements
14. Benefits
15. Conclusion

1. Executive Summary

Introduction

The Indoore Backend API Automation Framework is a scalable, modular, and enterprise-grade automation framework designed to validate REST APIs for the Indoore MDMS platform. The framework was created to improve test reliability, reduce manual validation effort, and provide comprehensive verification through schema validation, database validation, contract testing, and mutation-proof testing.

Unlike conventional API automation frameworks that primarily verify HTTP responses, this framework validates the complete backend behaviour by comparing API responses against database records, enforcing strict response contracts, and detecting regressions through mutation testing. The layered architecture promotes maintainability, reusability, and ease of extension, making it suitable for large-scale backend testing.

Purpose

The framework addresses common challenges in backend API testing, including inconsistent response validation, duplicated automation logic, manual database verification, and limited regression detection. It introduces a structured architecture that enables reusable components, centralized validation logic, and standardized testing practices across multiple functional modules.

The primary purpose of this framework is to ensure backend reliability while reducing maintenance effort and increasing confidence in production deployments.

2. Business Problem

Before this framework was implemented, backend API testing faced several limitations:

- Manual verification of API responses against database records.
- Repetitive validation logic across test cases.
- Lack of standardized schema validation.
- Difficulty identifying unintended API changes.
- High maintenance cost due to duplicated code.
- Limited confidence during regression testing.
- Inconsistent test implementation across modules.

These challenges increased testing effort and made it difficult to maintain consistent quality as the application expanded.

3. Framework Vision

The vision of the framework is to establish a standardized automation platform capable of supporting multiple backend modules while maintaining high reliability, consistency, and scalability.

Key Principles

- Reusable architecture
- Modular design
- Strong validation strategy
- Minimal code duplication
- Easy onboarding for new engineers
- Continuous maintainability

4. Objectives

The framework was designed to achieve the following objectives:

1. Standardize backend API automation.
2. Reduce repetitive implementation.
3. Increase regression confidence.
4. Validate API responses using strict schemas.
5. Cross-validate critical data with the database.
6. Detect breaking API changes through contract snapshots.
7. Identify validation weaknesses using mutation testing.
8. Support scalable module expansion.

5. Technology Stack

Category	Technology
Language	TypeScript
Framework	Playwright
Runtime	Node.js
Database	PostgreSQL
Schema Validation	Zod
Reporting	Allure
CI/CD	GitHub Actions
Package Manager	npm
Version Control	Git

6. Framework Overview

The framework follows a layered architecture where each responsibility is isolated into dedicated components. Every module follows a consistent structure consisting of API interaction, data preparation, response mapping, validation, database verification, and automated test execution.

This design promotes reusability and minimizes coupling between components, allowing new modules to be added with minimal effort while maintaining consistent implementation standards.

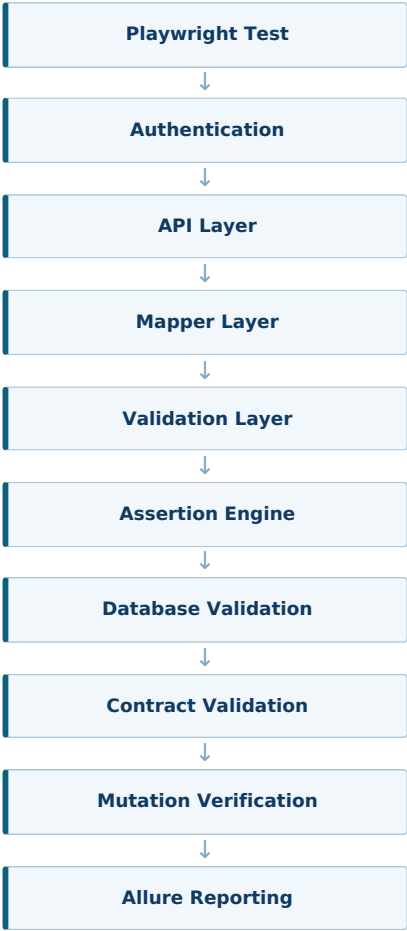
7. Architecture Principles

The framework is based on the following engineering principles:

Design Foundations • Separation of concerns • Single responsibility • Reusability • Scalability	Engineering Discipline • Consistency • Maintainability • Extensibility • Strong typing
Validation Philosophy • Validation-first design • Read-only database verification	

8. High-Level Architecture

The diagram below illustrates the end-to-end flow of a single test execution, from the Playwright test runner through authentication, request execution, validation, and reporting.



9. Layered Framework Design

The framework consists of seven primary layers:

Layer	Responsibility
API Layer	Handles HTTP communication with backend services.
Data Layer	Manages test data and request payloads.
Mapper Layer	Converts raw responses into structured objects.
Validator Layer	Performs schema and business-rule validation.
Database Layer	Executes read-only validation against PostgreSQL.
Schema Layer	Defines strict Zod response schemas.
Test Layer	Orchestrates end-to-end test execution.

10. Folder Structure

```
src/  
├─ core/  
├─ config/  
├─ fixtures/  
├─ modules/  
│   ├─ AUTH/  
│   ├─ MASTER-DATA/  
│   ├─ CONSUMERS/  
│   ├─ BILLING/  
│   ├─ REPORTS/  
│   └─ REVENUE-PROTECTION/  
│   └─ ...  
├─ scripts/  
├─ docs/  
└─ reports/
```

This standardized layout ensures that every module follows the same conventions, improving discoverability and maintainability.

11. Request Lifecycle

1. Playwright initiates the test.
2. Authentication establishes a valid session.
3. The API layer sends the request.
4. The mapper transforms the response.
5. Zod validates the response schema.
6. Business validators verify domain-specific rules.
7. Database validation confirms data integrity.
8. Contract snapshots detect structural changes.
9. Mutation tests evaluate validation robustness.
10. Results are reported through Allure.

12. Framework Features

Architecture & Reuse - Layered architecture - Reusable components - Centralized configuration - Modular scalability	Validation Strength - Strict schema validation - Database cross-validation - Contract snapshot testing - Mutation-proof testing
Execution & Reliability - Authentication management - Shared validation engine - Assertion engine - Retry mechanisms	Delivery - CI/CD integration - Rich reporting

13. Engineering Achievements

The framework delivers several engineering improvements:

- Consistent implementation across modules.
- Reduced duplication through shared components.
- Improved confidence in backend releases.
- Faster onboarding for new contributors.
- Simplified maintenance.
- Enhanced regression detection.
- Standardized validation strategy.
- Improved test readability.

14. Benefits

For Developers • Early detection of backend regressions. • Faster debugging. • Clear validation feedback.	For QA Engineers • Reusable framework components. • Reduced manual verification. • Consistent testing practices.
For Project Managers • Improved release confidence. • Reduced production defects. • Better visibility through reporting.	For Stakeholders • Higher software quality. • Increased reliability. • Lower maintenance costs.

15. Conclusion

The Indoore Backend API Automation Framework establishes a standardized, scalable, and maintainable approach to backend API testing. By combining layered architecture, strict schema validation, contract verification, mutation testing, and database cross-validation, it provides a robust foundation for validating complex backend systems.

This framework is designed not only to support current project requirements but also to accommodate future expansion with minimal architectural changes, making it a long-term solution for enterprise backend automation.

VOLUME 2

02

Core Framework Components

The shared engineering infrastructure — authentication, API, data, mapper, validation, assertion, database, schema, and resilience layers — that every module builds upon.

1. Core Framework Overview
2. Framework Component Architecture
3. Authentication Engine
4. API Layer
5. Data Layer
6. Mapper Layer
7. Validation Engine
8. Assertion Engine
9. Database Validation Layer
10. Schema Layer (Zod)
11. Shared Utilities
12. Retry & Resilience Engine
13. Error Handling Strategy
14. Logging & Reporting
15. Design Patterns
16. Engineering Benefits
17. Conclusion

1. Core Framework Overview

Purpose

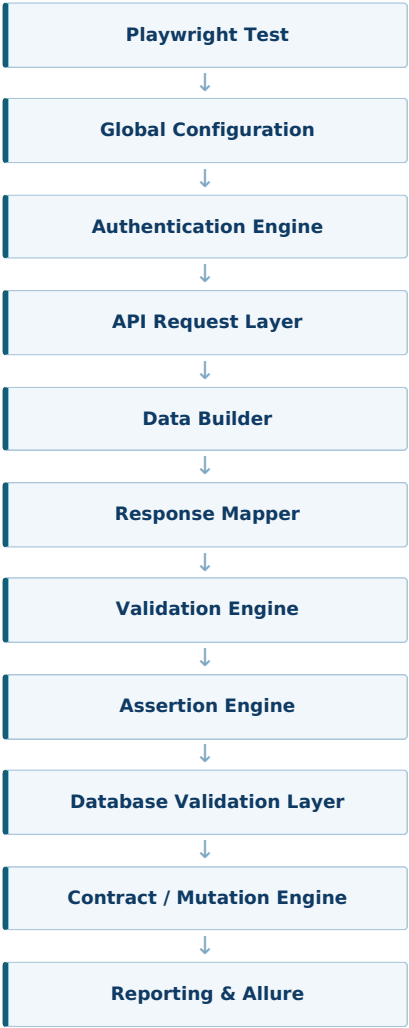
The Core Framework forms the foundation of the Indoore Backend API Automation Framework. It provides reusable infrastructure that supports every automation module while ensuring consistency, scalability, and maintainability.

Instead of implementing authentication, validation, reporting, and error handling independently within each module, these concerns are centralized into shared components. This architecture minimizes duplication, simplifies maintenance, and allows new modules to adopt the same engineering standards with minimal effort.

Design Goals

- Reusability
- Separation of Concerns
- Strong Typing
- Configurability
- Scalability
- Fault Tolerance
- Maintainability
- Consistent Validation

2. Framework Component Architecture



Each component has a clearly defined responsibility and communicates through well-defined interfaces.

3. Authentication Engine

Overview

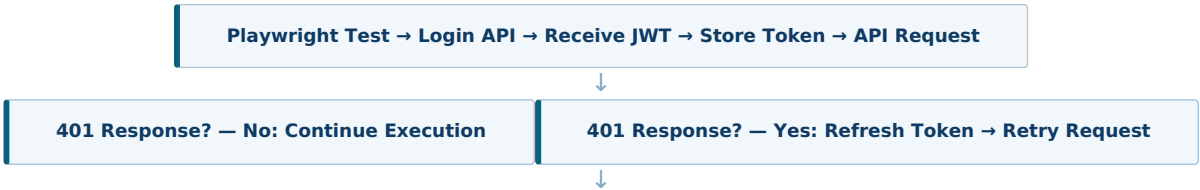
The Authentication Engine manages secure communication with backend APIs by handling login, token generation, refresh logic, and authorization headers.

This component ensures that tests remain focused on business validation rather than authentication mechanics.

Responsibilities

- User Authentication
- JWT Token Retrieval
- Token Refresh
- Session Reuse
- Header Construction
- Authorization Validation
- Login Failure Handling

Authentication Flow



Benefits

- Eliminates duplicate login code.
- Reduces authentication failures.
- Supports long-running test suites.
- Simplifies maintenance.

4. API Layer

Overview

The API Layer serves as the communication gateway between the automation framework and backend services. Each module exposes API classes responsible for constructing requests, managing headers, sending HTTP calls, and returning typed responses.

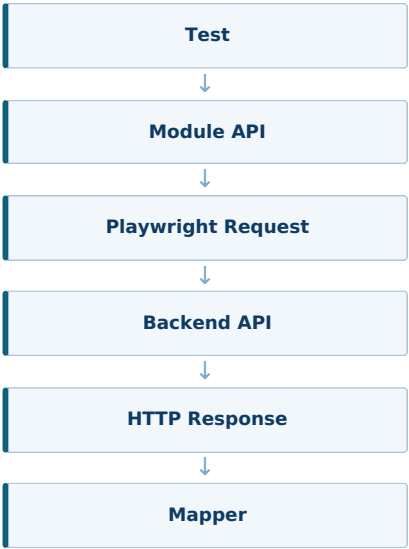
Responsibilities

- Endpoint Management
- HTTP Request Execution
- Query Parameter Handling
- Payload Submission
- Authentication Header Injection
- Response Retrieval

Design Principles

- Single Responsibility
- Reusable Methods
- Strong Typing
- Minimal Business Logic

Example Workflow



5. Data Layer

The Data Layer manages test inputs, request payloads, and reusable datasets. It separates test data from business logic, improving readability and maintainability.

Responsibilities

- Payload Builders
- Dynamic Data
- Static Test Data
- Boundary Values
- Negative Inputs
- Test Fixtures

Advantages

- Reusable payloads
- Easier maintenance
- Reduced duplication
- Cleaner test cases

6. Mapper Layer

Raw API responses are converted into structured domain models using the Mapper Layer. This abstraction isolates response parsing from validation logic.

Responsibilities

- Response Mapping
- Field Transformation
- Nullable Handling
- Data Normalization
- DTO Creation

Benefits

- Cleaner Validators
- Consistent Models
- Simplified Assertions

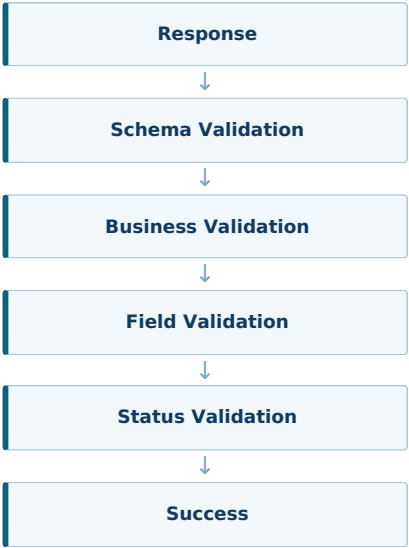
7. Validation Engine

The Validation Engine centralizes all response validation logic. Rather than embedding assertions within tests, validation rules are implemented as reusable methods that verify schema integrity, business rules, and data consistency.

Responsibilities

- Response Validation
- Business Rules
- Pagination Validation
- Required Fields
- Type Validation
- Value Validation
- Edge Cases

Validation Flow



Engineering Advantages

- Reusable validation
- Consistent assertions
- Easier maintenance
- Reduced duplication

8. Assertion Engine

The Assertion Engine provides a unified interface for verifying test outcomes. It standardizes assertions across modules and produces consistent error messages.

Responsibilities

- Equality Checks
- Collection Validation
- Performance Assertions
- Error Assertions
- Null Validation
- Boundary Validation

Benefits

- Uniform assertions
- Better diagnostics
- Cleaner test code

9. Database Validation Layer

API responses are validated against PostgreSQL using read-only SQL queries. Database validation increases confidence that backend services return accurate and complete information.

Responsibilities

- SQL Execution
- Record Verification
- Count Validation
- Sample Comparison
- Business Rule Validation

Validation Process



Benefits

- Detect backend inconsistencies
- Validate business data
- Improve regression confidence

10. Schema Layer (Zod)

The Schema Layer defines strict API response contracts using Zod. Schemas validate the structural integrity of API responses before business validations execute.

Responsibilities

- Response Structure Validation
- Required Fields
- Optional Fields
- Nullable Handling
- Type Enforcement
- Strict Object Validation

Why Zod?

- Type Safety
- Runtime Validation
- Clear Error Messages
- Strong TypeScript Integration

11. Shared Utilities

The framework includes reusable utilities shared across all modules.

Configuration Manager • Environment handling • URLs • Credentials	Date Utilities • Date formatting • Time calculations
Random Data Generator • Dynamic payloads • Unique identifiers	Helper Functions • Common transformations • Utility methods
Constants • Shared configuration • Enumerations • Default values	

12. Retry & Resilience Engine

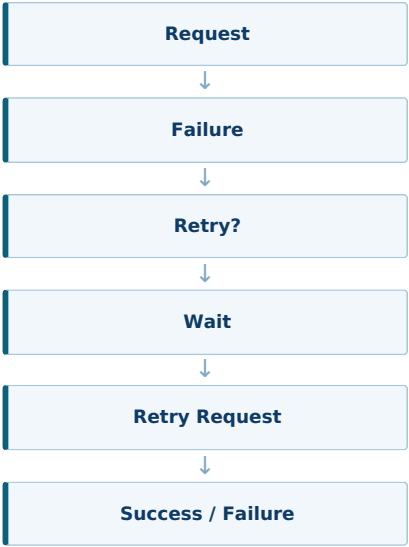
Purpose

Temporary backend failures should not immediately fail test execution. The Retry Engine automatically retries recoverable operations.

Retry Conditions

- Network Failure
- Gateway Timeout
- Service Unavailable
- Connection Reset
- Token Expiry

Retry Flow



Benefits

- Stable execution
- Reduced flaky tests
- Better CI reliability

13. Error Handling Strategy

The framework categorizes failures into:

Category	Description
Validation Errors	Incorrect API responses.
Authentication Errors	Expired or invalid tokens.
Network Errors	Connectivity failures.
Database Errors	SQL execution issues.
Business Rule Failures	Incorrect business logic.
Contract Violations	Unexpected API changes.

Each category produces descriptive error messages to simplify troubleshooting.

14. Logging & Reporting

The framework records execution details at each stage.

Logged Information

- Request URL
- HTTP Method
- Headers
- Payload
- Response Time
- Status Code
- Validation Results
- Database Comparison
- Error Details

Reporting

Integrated with Allure to provide:

- Execution Summary
- Pass/Fail Metrics
- Response Attachments
- Error Screens
- Performance Information

15. Design Patterns

The framework adopts several established software design principles.

Pattern	Application
Factory Pattern	Used for request data creation.
Builder Pattern	Used for payload construction.
Strategy Pattern	Supports interchangeable validation strategies.
Singleton Pattern	Configuration and shared services.
Layered Architecture	Separates responsibilities across components.

These patterns improve modularity and make the framework easier to extend.

16. Engineering Benefits

The Core Framework delivers measurable improvements:

- Consistent implementation across modules
- Reduced code duplication
- Faster onboarding
- Easier maintenance
- Improved scalability
- Better reliability
- Strong validation
- Standardized reporting
- Simplified debugging
- Enterprise-ready architecture

17. Conclusion

The Core Framework establishes the reusable infrastructure upon which all backend API automation modules are built. By centralizing authentication, API communication, validation, assertions, database verification, retry mechanisms, and reporting, it enables a scalable and maintainable automation ecosystem.

This architecture allows new modules to be developed using consistent engineering standards while reducing duplication and improving long-term maintainability. As the framework evolves, these shared components provide a stable foundation for extending automation coverage without requiring architectural redesign.

VOLUME 3

03

Four Quality Pillars

Schema validation, contract snapshot testing, mutation-proof testing, and database cross-validation — the layered strategy that gives the framework its depth of confidence.

1. Introduction
2. Why Four Quality Pillars?
3. Quality Architecture
4. Pillar 1 – Zod Schema Validation
5. Pillar 2 – Contract Snapshot Testing
6. Pillar 3 – Mutation-Proof Testing
7. Pillar 4 – Database Cross-Validation
8. Combined Validation Workflow
9. Framework Integration
10. Benefits & Outcomes
11. Best Practices
12. Conclusion

1. Introduction

A reliable API automation framework should verify more than HTTP status codes. Modern backend services evolve continuously, and a passing status code alone cannot guarantee that an endpoint still behaves correctly.

To address this, the Indoore Backend API Automation Framework adopts four complementary validation layers — referred to as the Four Quality Pillars. Each pillar focuses on a different aspect of API quality, and together they provide broader confidence than any single validation approach.

1. Schema Validation
2. Contract Snapshot Testing
3. Mutation-Proof Testing
4. Database Cross-Validation

Each can detect a different class of defect.

2. Why Four Quality Pillars?

Traditional API tests often verify:

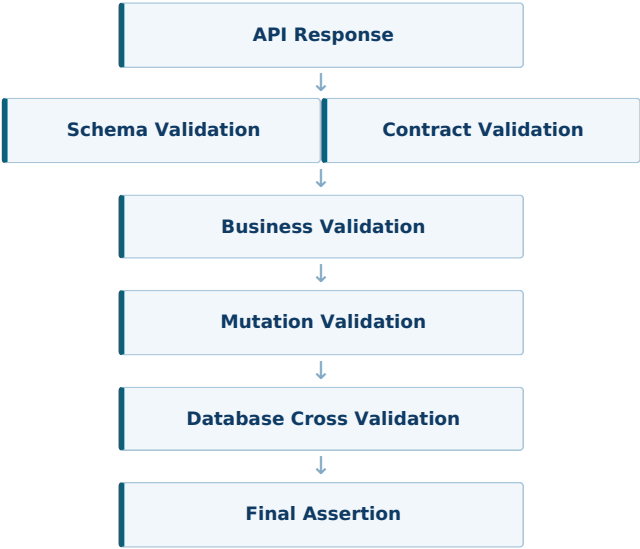
- HTTP status code
- Response time
- A few response fields

This leaves several risks:

Risk	Example
Response structure changes	New field removed
Wrong business value	Incorrect consumer count
Database mismatch	API returns stale data
Weak validators	Invalid payload still passes
Silent regression	Existing tests continue to pass

The Four Quality Pillars address these risks by validating the response from multiple perspectives.

3. Quality Architecture



4. Pillar 1 — Zod Schema Validation

Purpose

Schema validation ensures that every API response adheres to a predefined structure before business logic is evaluated. This is the first quality gate executed after a response is received.

Objectives

- Validate response structure
- Detect missing fields
- Verify data types
- Handle nullable values
- Enforce required properties
- Detect unexpected API changes

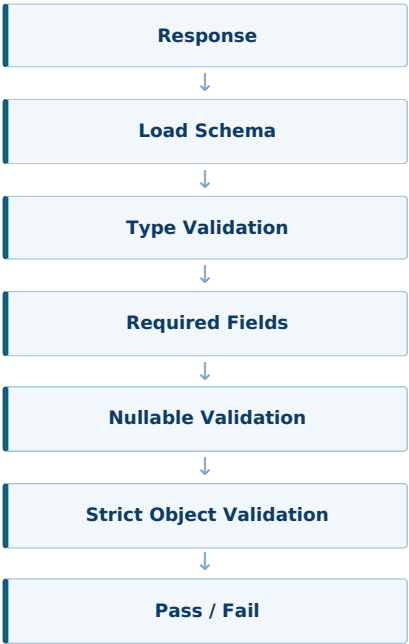
Implementation

Each module maintains dedicated schema definitions:

```
AUTH/  
  schemas/  
  
MASTER-DATA/  
  schemas/  
  
REPORTS/  
  schemas/  
  
REVENUE-PROTECTION/  
  schemas/
```

Schemas remain close to the business domain, making them easier to maintain.

Validation Flow



Engineering Benefits

- Early defect detection
- Runtime validation
- Strong TypeScript integration
- Reduced downstream failures

Example

The API accidentally renames `consumerName` to `consumer_name`. The schema immediately detects the structural mismatch before business assertions execute.

5. Pillar 2 — Contract Snapshot Testing

Purpose

Contract snapshots capture the expected API response structure and allow future executions to detect unintended changes. Instead of manually inspecting responses after each backend release, snapshots provide an automated mechanism for identifying structural regressions.

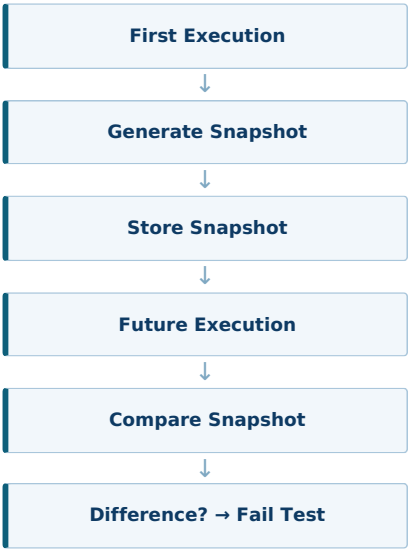
Why Contracts?

Back-end teams may introduce changes such as:

- Field removal
- Renamed properties
- Nesting changes
- Response envelope modifications

These changes can break clients without affecting HTTP status codes.

Snapshot Lifecycle



Benefits

- Detect API evolution
- Improve release confidence
- Simplify regression testing
- Support controlled contract updates

Example

Previous response contained `consumerId`, `consumerName`, `status`. The new response drops `consumerName`. Snapshot comparison identifies the removed field immediately.

6. Pillar 3 — Mutation-Proof Testing

Purpose

Mutation testing validates the quality of the test suite itself. Rather than asking “Does the API pass?”, mutation testing asks “Would our validators fail if the API returned incorrect data?”

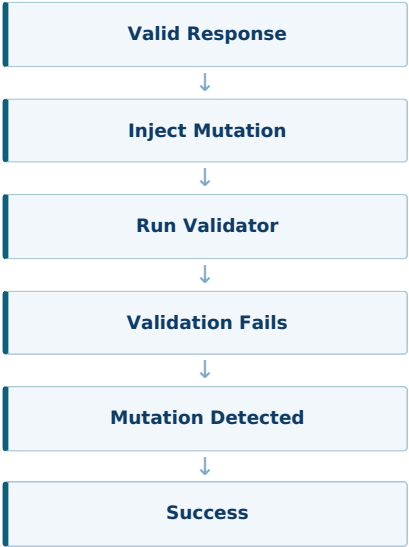
Philosophy

A test suite that always passes may still be ineffective if it cannot detect realistic defects. Mutation testing deliberately introduces controlled faults into test data or responses to verify that validation logic reacts correctly.

Typical Mutations

- Missing required field
- Incorrect data type
- Invalid enum
- Negative identifier
- Empty string
- Null value
- Duplicate entry
- Unexpected property

Workflow



Benefits

- Stronger validation
- Reduced false confidence
- Better regression protection
- Improved validator quality

Example

Original response has `consumerId: 25`. The mutation changes it to `consumerId: "ABC"`. A correct validator rejects the response.

7. Pillar 4 — Database Cross-Validation

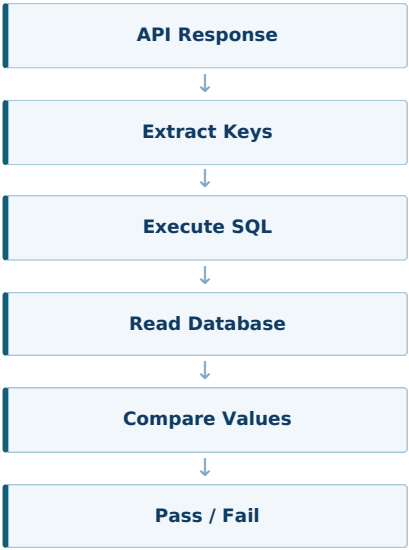
Purpose

API responses are validated against the underlying PostgreSQL database to ensure that business data is accurate. This is especially valuable for reporting, dashboards, analytics, and aggregation endpoints.

Validation Scope

- Record counts
- Individual fields
- Aggregated values
- Lookup values
- Status information
- Pagination totals

Workflow



Benefits

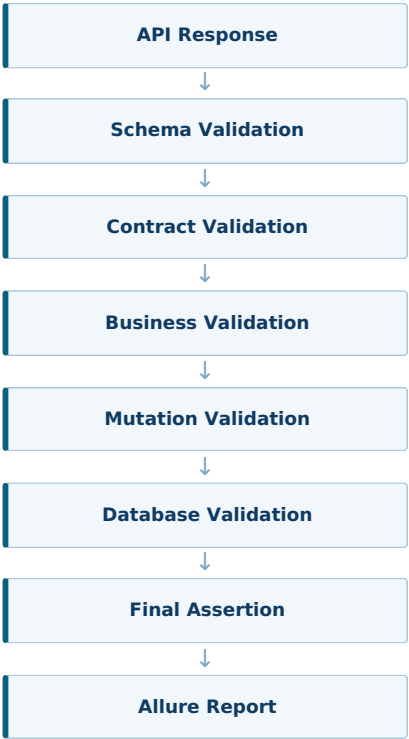
- Detect stale responses
- Identify backend calculation issues
- Validate business logic
- Increase release confidence

Example

The API reports `consumerCount = 112` while the database shows `consumerCount = 98`. The mismatch is reported, prompting investigation into the API or underlying query.

8. Combined Validation Workflow

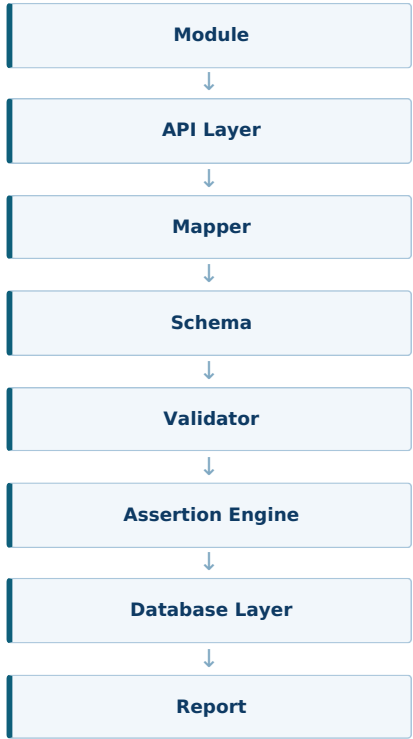
The framework applies the pillars in sequence so that each stage builds on the previous one.



This layered approach helps isolate failures quickly. Structural issues are identified before business-rule checks, and database comparisons are only performed once the response format is known to be valid.

9. Framework Integration

Each module follows the same validation pipeline:



Using a consistent workflow across modules makes the framework easier to extend and simplifies onboarding for future contributors.

10. Benefits & Outcomes

Technical Benefits • Consistent response validation • Stronger regression detection • Improved test reliability • Better defect isolation • Higher confidence in releases	Operational Benefits • Reduced manual verification • Faster troubleshooting • Easier framework maintenance • Consistent implementation standards
--	---

11. Best Practices

To maintain the effectiveness of the Four Quality Pillars:

1. Keep schemas aligned with the latest API contracts.
2. Review and intentionally update snapshots when approved API changes occur.
3. Add new mutation cases whenever new validation rules are introduced.
4. Use read-only SQL for database validation to avoid modifying production data.
5. Keep validation logic modular and reusable across modules.
6. Document any intentional exceptions to validation rules.

12. Conclusion

The Four Quality Pillars form the quality backbone of the Indoore Backend API Automation Framework. Rather than relying on simple response assertions, the framework validates API behaviour through structural verification, contract consistency, mutation resistance, and database integrity checks.

This multi-layered strategy increases confidence in backend releases, improves defect detection, and provides a repeatable approach to validating complex enterprise APIs. It also establishes a consistent validation standard that can be applied across existing and future modules with minimal architectural change.

VOLUME 4

04

Module Architecture & Module Catalogue

The standard module template, development lifecycle, and a full catalogue of the nineteen business-domain modules implemented across the platform.

1. Module Architecture Overview
2. Standard Module Design
3. Module Development Lifecycle
4. Common Folder Structure
5. Module Implementation Standards
6. Module Catalogue
7. Module Dependency Matrix
8. Module Coverage Summary
9. Engineering Metrics
10. Best Practices
11. Future Expansion Strategy
12. Conclusion

1. Module Architecture Overview

Purpose

The Indoore Backend API Automation Framework follows a module-driven architecture, where each business domain is implemented as an independent automation module. This design isolates responsibilities, enables parallel development, and allows modules to evolve without impacting unrelated functionality.

Each module adheres to the same engineering standards, including:

- Consistent folder structure
- Shared framework components
- Reusable validation
- Centralized authentication
- Standardized reporting
- Database verification
- Four Quality Pillars

By enforcing a common implementation pattern, the framework remains maintainable as the number of APIs and business domains grows.

2. Standard Module Design

Every module follows a layered architecture.

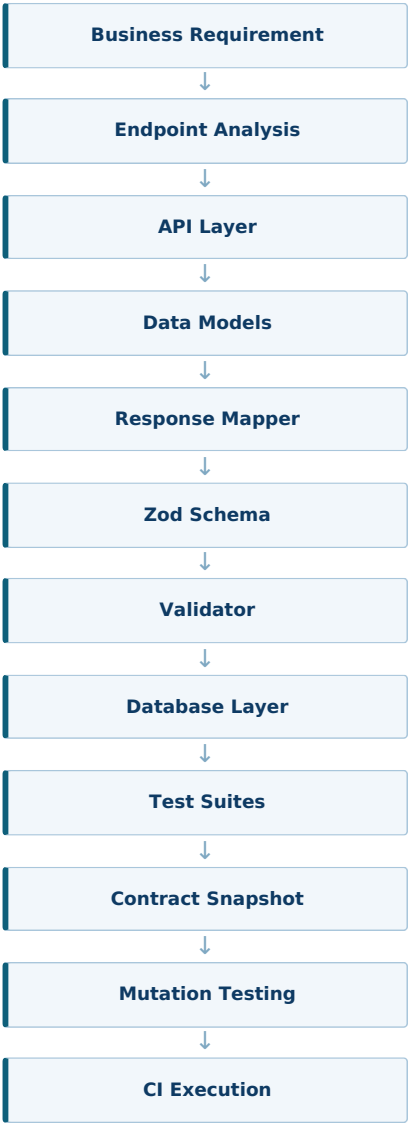
```
MODULE
├─ Api
├─ Data
├─ Mapper
├─ Validator
├─ Db
├─ Schemas
├─ Tests
└─ Fixtures
```

Responsibilities

Layer	Responsibility
API	Sends HTTP requests and handles responses
Data	Request payloads, fixtures, dynamic test data
Mapper	Converts raw API responses into domain objects
Validator	Business-rule and response validation
DB	Read-only SQL validation and API-to-DB comparison
Schemas	Zod response contracts
Tests	Positive, negative, authentication, contract, mutation, and DB test suites

3. Module Development Lifecycle

Each new module is implemented using a repeatable engineering workflow.



This lifecycle ensures that every module follows the same quality standards before integration into the framework.

4. Common Folder Structure

```
src/  
└─ modules/  
    └─ MODULE_NAME/  
        ├── Api/  
        ├── Data/  
        ├── Db/  
        ├── Mapper/  
        ├── Schemas/  
        ├── Validator/  
        ├── Tests/  
        ├── Fixtures/  
        └─ README.md
```

Each directory has a clearly defined responsibility, making the framework easier to navigate and extend.

5. Module Implementation Standards

Every module should include:

Implementation Layer • API classes • Typed request models • Typed response models • Zod schemas	Validation Layer • Business validators • Assertion integration • Database validation
Quality Layer • Contract snapshots • Mutation tests • Authentication tests	Coverage & Reporting • Positive and negative scenarios • Allure reporting support

6. Module Catalogue

The following catalogue summarises the primary automation modules implemented within the framework.

6.1 AUTH Module

Validates authentication workflows, user sessions, token generation, and authorization.

Coverage: Login, Logout, JWT validation, Token refresh, Unauthorized access, Invalid credentials, Session expiry

Layers: API, Validator, Schemas, Tests

6.2 MASTER DATA Module

Validates master entities used throughout the MDMS platform.

Coverage: Circle, Division, Subdivision, Feeder, DTR, Meter, Consumer master lookups

Validation: Schema, Pagination, Search, Hierarchy, Database comparison

6.3 CONSUMERS Module

Automates consumer-related APIs.

Coverage: Search, Consumer details, Pagination, Filters, Business validation, Error scenarios

Database Validation: Consumer existence, Record counts, Lookup validation

6.4 USERS Module

Validates user administration and profile management.

Coverage: User creation, User update, Profile retrieval, Permissions, Role assignment, Profile image APIs

6.5 PERMISSIONS Module

Verifies role-based access control.

Coverage: Roles, Permissions, Mapping, Access validation, Authorization

6.6 BILLING Module

Automates billing and revenue-related APIs.

Coverage: Billing summaries, Billing history, Revenue reports, Consumer billing details

6.7 CONSUMPTION Module

Validates energy consumption APIs.

Coverage: Daily consumption, Monthly consumption, Trend analysis, Consumption statistics

6.8 REPORTS Module

Automates reporting APIs.

Coverage: Report generation, Pagination, Filtering, Summary reports

6.9 MIS Module

Validates Management Information System APIs.

Coverage: Dashboard, KPIs, Summary APIs, Communication statistics, Event metrics

6.10 HES COMMANDS Module

Automates asynchronous Head-End System command workflows.

Coverage: Command initiation, Status polling, Completion verification, Failure handling, Timeout handling

Special Capability: Intelligent polling strategy

6.11 ASSET MANAGEMENT Module

Validates electrical asset inventory.

Coverage: Asset hierarchy, Asset details, Consumer linkage, Asset statistics

Quality Features: Contract snapshots, Mutation testing, Database verification

6.12 REVENUE PROTECTION Module

Automates revenue protection analytics and aberration APIs.

Coverage: Aberration entries, Dashboard APIs, Filtering, Pagination, Database validation

Validation: Business rules, Data quality, Aggregation checks

6.13 COMMERCIAL ANALYSIS Module

Validates commercial analysis APIs.

Coverage: Commercial KPIs, Reports, Aggregated analytics, Trend validation

6.14 AUDIT LOGS Module

Automates audit trail validation.

Coverage: User activity, Change history, Filters, Search, Pagination

6.15 NOTIFICATIONS Module

Validates notification delivery APIs.

Coverage: Notification history, Status, Delivery metadata

6.16 OVERALL DASHBOARD Module

Automates platform-wide dashboard metrics.

Coverage: Dashboard widgets, KPI validation, Summary statistics

6.17 DTR Module

Validates Distribution Transformer APIs.

Coverage: DTR details, Communication, Meter mapping, Capacity information

6.18 ENERGY AUDITS Module

Automates energy audit APIs.

Coverage: Energy balance, Loss calculations, Validation reports

6.19 METER REPLACEMENT Module

Automates meter replacement workflows.

Coverage: Consumer lookup, Meter validation, Submission history, Dashboard summary, Replacement eligibility

7. Module Dependency Matrix

Component	Shared Across Modules
Authentication	✓
API Layer	✓
Mapper	✓
Validation Engine	✓
Assertion Engine	✓
Retry Engine	✓
Database Layer	✓
Zod Schemas	✓
Contract Testing	✓
Mutation Testing	✓
Reporting	✓

This shared foundation ensures consistency while allowing each module to focus on its specific business logic.

8. Module Coverage Summary

Category	Coverage
Authentication	✓
Master Data	✓
Consumers	✓
Users	✓
Permissions	✓
Billing	✓
Consumption	✓
Reports	✓
MIS	✓
HES Commands	✓
Revenue Protection	✓
Commercial Analysis	✓
Asset Management	✓
Audit Logs	✓
Notifications	✓
Dashboard	✓
DTR	✓
Energy Audits	✓
Meter Replacement	✓

Coverage Note

This table reflects the modules discussed and implemented in the framework documentation. As additional business domains are added, the catalogue should be updated to maintain a current inventory.

The modular architecture provides several operational benefits:

- Independent module execution
- Reusable framework components
- Consistent validation pipeline
- Reduced maintenance effort
- Simplified onboarding
- Parallel development support
- Standardised reporting
- Scalable extension model

These characteristics help the framework grow without requiring significant architectural changes.

10. Best Practices

When creating or extending a module:

1. Follow the standard folder structure.
2. Keep business logic within validators rather than API classes.
3. Define Zod schemas before implementing business validations.
4. Reuse shared framework utilities where possible.
5. Include positive, negative, contract, mutation, and database validation tests as appropriate.
6. Maintain consistent naming conventions and documentation.

11. Future Expansion Strategy

The architecture is designed to support additional modules with minimal framework changes. The recommended onboarding workflow for a new module is:

1. Scaffold the standard folder structure.
2. Implement API interactions.
3. Add request and response models.
4. Define Zod schemas.
5. Build validators.
6. Implement database validation (where applicable).
7. Add contract snapshot tests.
8. Add mutation-proof tests.
9. Integrate with CI/CD and reporting.

This approach promotes consistency across all business domains.

12. Conclusion

The module-driven architecture enables the Indoore Backend API Automation Framework to scale across multiple business domains while maintaining a consistent implementation model. Shared infrastructure, standardized validation practices, and reusable components reduce duplication and simplify long-term maintenance.

As new APIs and services are introduced, they can be integrated by following the established module template, ensuring that the framework continues to evolve without compromising engineering standards.

VOLUME 5

05

CI/CD, Test Execution Pipeline & Engineering Operations

Environment management, branching strategy, the GitHub Actions pipeline, observability, and the release-readiness gates that keep the framework operating reliably.

1. Engineering Operations Overview
2. Test Execution Lifecycle
3. Environment Management
4. Git Workflow & Branching Strategy
5. CI/CD Architecture
6. GitHub Actions Pipeline
7. Test Execution Strategy
8. Reporting & Observability
9. Logging & Diagnostics
10. Performance Monitoring
11. Failure Classification & Recovery
12. Release Readiness
13. Operational Best Practices
14. Future Enhancements
15. Conclusion

1. Engineering Operations Overview

Purpose

A robust automation framework extends beyond writing test cases. Operational practices ensure that tests execute consistently across developer machines and continuous integration environments while producing actionable feedback.

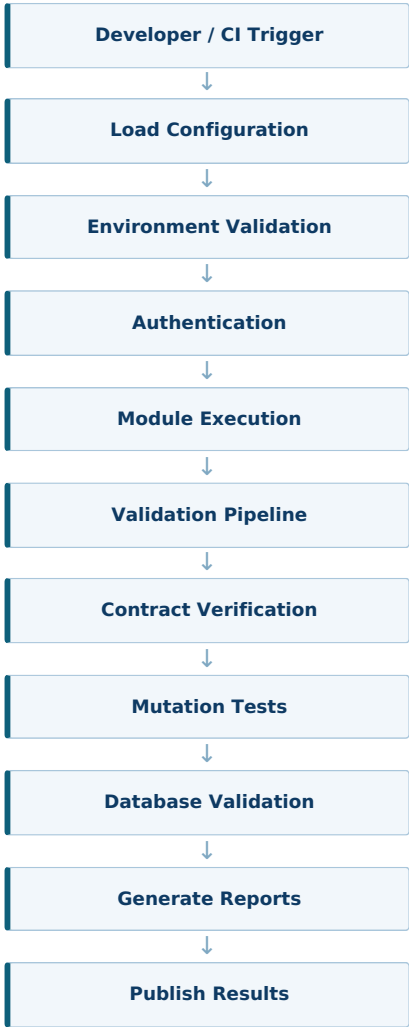
This volume describes how the Indoore Backend API Automation Framework supports automated execution, reporting, diagnostics, and release validation.

Operational Goals

- Consistent execution
- Repeatable builds
- Automated quality gates
- Reliable reporting
- Fast defect identification
- Scalable execution
- Easy maintenance

2. Test Execution Lifecycle

Every test execution follows a structured lifecycle.



Each stage produces logs and metrics that help diagnose failures.

3. Environment Management

The framework supports multiple execution environments through centralized configuration.

Supported Environments

- Local Development
- QA
- UAT
- Staging
- Production (read-only validation only)

Configuration Management

Configuration includes:

- Base URL
- Authentication credentials
- API tokens
- Database connection
- Timeouts
- Retry policies
- Feature flags

```
.env  
.env.qa  
.env.uat  
.env.staging
```

Centralising configuration reduces duplication and simplifies environment switching.

4. Git Workflow & Branching Strategy

The framework follows a controlled branching model to support parallel development and stable releases.

```
main
├─ QA
├─ feature/auth
├─ feature/master-data
├─ feature/revenue-protection
└─ hotfix/*
```

Workflow

1. Create feature branch.
2. Develop and test locally.
3. Merge into QA.
4. Execute regression suite.
5. Review reports.
6. Merge into main after approval.

5. CI/CD Architecture

Continuous Integration automates test execution whenever changes are introduced.



This pipeline ensures that regressions are detected early and consistently.

6. GitHub Actions Pipeline

Typical Pipeline Stages

Stage	Activities
1 — Checkout Source	Retrieve repository; restore branch state
2 — Dependency Installation	Install Node modules; validate package integrity
3 — Build	Compile TypeScript; static analysis; configuration validation
4 — Execute Tests	Authentication, API tests, contract tests, mutation tests, database validation
5 — Reporting	Generate Allure results; archive logs; upload reports

7. Test Execution Strategy

Tests are organised to maximise coverage while keeping execution manageable.

Category	Description
Smoke Tests	Validate critical functionality.
Regression Tests	Execute the complete suite.
Module Tests	Focus on a single business domain.
Contract Tests	Detect API contract changes.
Mutation Tests	Validate the effectiveness of validators.
Database Tests	Verify API responses against database records.
Performance Checks	Record response times for trend analysis.

Parallel Execution

Where appropriate, modules execute in parallel to reduce overall runtime while maintaining isolation between test data and environments.

8. Reporting & Observability

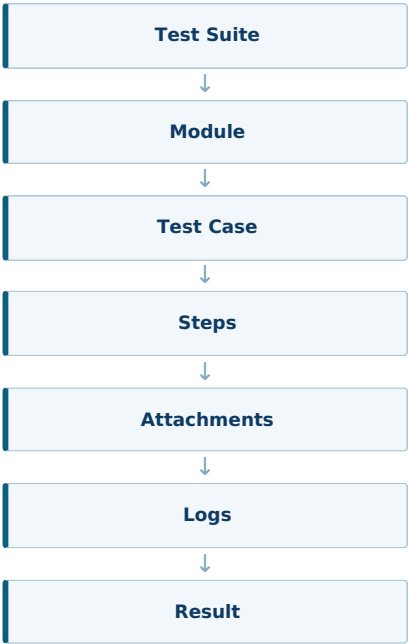
The framework generates detailed execution reports for developers, QA engineers, and project stakeholders.

Allure Reporting

Captured information includes:

- Test status
- Execution duration
- Request and response details
- Validation results
- Attachments
- Error messages
- Environment information

Report Structure



This hierarchy helps identify failing scenarios quickly.

9. Logging & Diagnostics

Comprehensive logging supports troubleshooting.

Logged Information

- Request URL
- HTTP method
- Headers
- Request payload
- Response payload
- Response time
- Authentication events
- SQL validation results
- Contract comparison results
- Mutation outcomes
- Exceptions

Logging Levels

Level	Purpose
INFO	Normal execution
WARN	Recoverable issues
ERROR	Test failures
DEBUG	Detailed troubleshooting

10. Performance Monitoring

Performance is monitored to identify regressions over time.

Captured Metrics

- Response time
- Request duration
- Retry count
- Timeout frequency
- Success rate
- Failure rate

These metrics provide operational visibility and support trend analysis across builds.

11. Failure Classification & Recovery

To simplify troubleshooting, failures are grouped into categories.

Failure Type	Description
Authentication	Login or token issues
API	HTTP errors or unexpected responses
Schema	Contract or type violations
Validation	Business-rule failures
Database	API-to-DB mismatches
Network	Connectivity issues
Configuration	Missing or invalid environment settings
Infrastructure	CI or runtime problems

Clear categorisation helps engineers identify the source of failures more quickly.

12. Release Readiness

Before a release, the following validation activities are recommended:

1. Smoke suite execution
2. Full regression suite
3. Contract verification
4. Mutation-proof execution
5. Database validation
6. Performance review
7. Critical defect review
8. Report generation

Quality Gate

A release should proceed only after the defined quality gates are satisfied.

13. Operational Best Practices

To maintain framework reliability:

- Keep environment configuration under version control where appropriate (excluding secrets).
- Rotate credentials securely.
- Review contract snapshots after approved API changes.
- Monitor long-running tests for optimisation opportunities.
- Keep module dependencies independent.
- Archive reports for historical comparison.
- Review flaky tests and address underlying causes.

14. Future Enhancements

The operational architecture can be extended with additional capabilities, such as:

- Automated quality dashboards
- Historical trend analysis
- API coverage metrics
- Performance benchmarking
- Notification integration (email or messaging platforms)
- Scheduled nightly regression execution
- Distributed test execution
- Self-healing retry strategies
- Advanced analytics for execution history

These enhancements can be adopted incrementally without changing the core framework architecture.

15. Conclusion

The engineering operations model complements the framework architecture by providing a structured approach to execution, reporting, diagnostics, and release validation. Automated pipelines, centralised configuration, comprehensive reporting, and layered quality gates help ensure that API automation remains reliable as the application and test suite grow.

Operational discipline is as important as test implementation. By combining sound engineering practices with a modular automation framework, the project supports repeatable, maintainable, and scalable backend quality assurance.